Programación Avanzada
IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha

Anuncios

12 de marzo de 2026

1. Si no lo han hecho, ¡respondan el formulario y acepten la invitación a Github!
2. Ya deberían tener todo instalado en su computador.
3. Hoy haremos la Actividad 1, qué nos servirá para acostumbrarnos al flujo de actividades del semestre.

Guía de estilo: PEP8

Guía de estilo

- ¿Qué es una guía de estilo?
- ¿Por qué usarla?
- Guía de estilo en Python: PEP 8.

Ejemplo de cambio de estilo ¿Qué cambia en el formato?

Antes de PEP8

```
def mifuncion(nombre,apellido):
    print(nombre+" "+apellido)
    print(time.today()       )

listaDeProfes = [["Jaime", "Middleton"],
["Pablo", "Araneda"],["Antonio",
"Lopez"],["Tamara", "Vidal"]
["Daniela", "Concha"]]

for lista in listaDeProfes:
  mifuncion(lista[0],

              lista[1])

import time
```

Después de PEP8

```
import time

def mi_funcion(nombre, apellido):
    print(nombre + " " + apellido)
    print(time.today())

lista_de_profes = [
    ["Jaime", "Middleton"],
    ["Pablo", "Araneda"],
    ["Antonio", "Lopez"],
    ["Tamara", "Vidal"],
    ["Daniela", "Concha"]
]

for lista in lista_de_profes:
    mi_funcion(lista[0], lista[1])
```

Modularización

Modularización

- ¿Qué es modularización?
- Cómo importar en Python
- ¿Por qué modularizar?

Ejemplo de modularización

modulo.py

```python
def funcion():
    print("Hola amiguitos de IIC2233")

def funcion2():
    print("Adios amiguitos de IIC2233")
```

main.py

```python
import modulo

import modulo as p

from modulo import funcion, funcion2

from modulo import * # Usted NO lo haga!!
```

Paths

Paths

- *Paths* absolutos y relativos
- *Paths* según el sistema operativo
- Librerías para manejo de *paths* :
  - OS
  - pathlib

Ejemplo de *paths*

Usted **NO** lo haga

```
p = “C:/Usuario/gatito/main.py” # absoluto

p = “/Usuario/gatito/main.py” # absoluto

p = “carpeta/tarea/main.py” # Puede fallar

p = “carpeta\tarea\main.py” # Puede fallar
```

Usted **SÍ** lo haga

```
import os
from pathlib import Path


p = os.path.join(“carpeta”, “tarea”, “main.py”)

p = Path("data/archivo.txt")
```

Uso de terminal

Comandos de terminal

```
comando argumentos* -opciones*
```

Windows:

- `cd` Ver y cambiar directorio actual
- `dir` Listar contenido del directorio
- `mkdir` Crear un directorio nuevo
- `echo` Mostrar mensaje en la terminal
- `>` Guardar contenido en un archivo

```
echo "contenido" > nombre.extension
```

Linux y macOS:

- `pwd` Ruta absoluta actual
- `cd` Ver y cambiar directorio actual
- `ls` Listar contenido del directorio
- `mkdir` Crear un directorio nuevo
- `touch` Crear un archivo nuevo

```
touch nombre.extension
```

Git

Flujo de git

Mi computador

Lista de cambios

Repositorio local

GitHub

`git add`

`git commit`

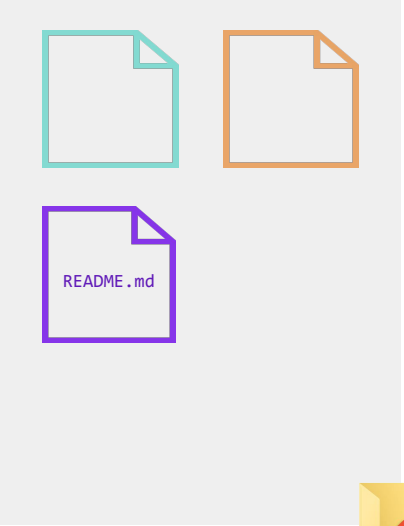
`git push`

Todo esto ocurre localmente

Internet

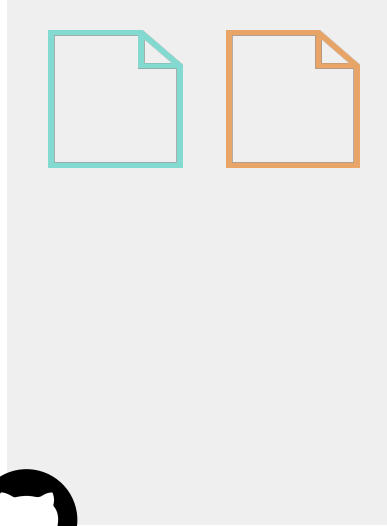
¿Qué acabamos de hacer?

Mi computador

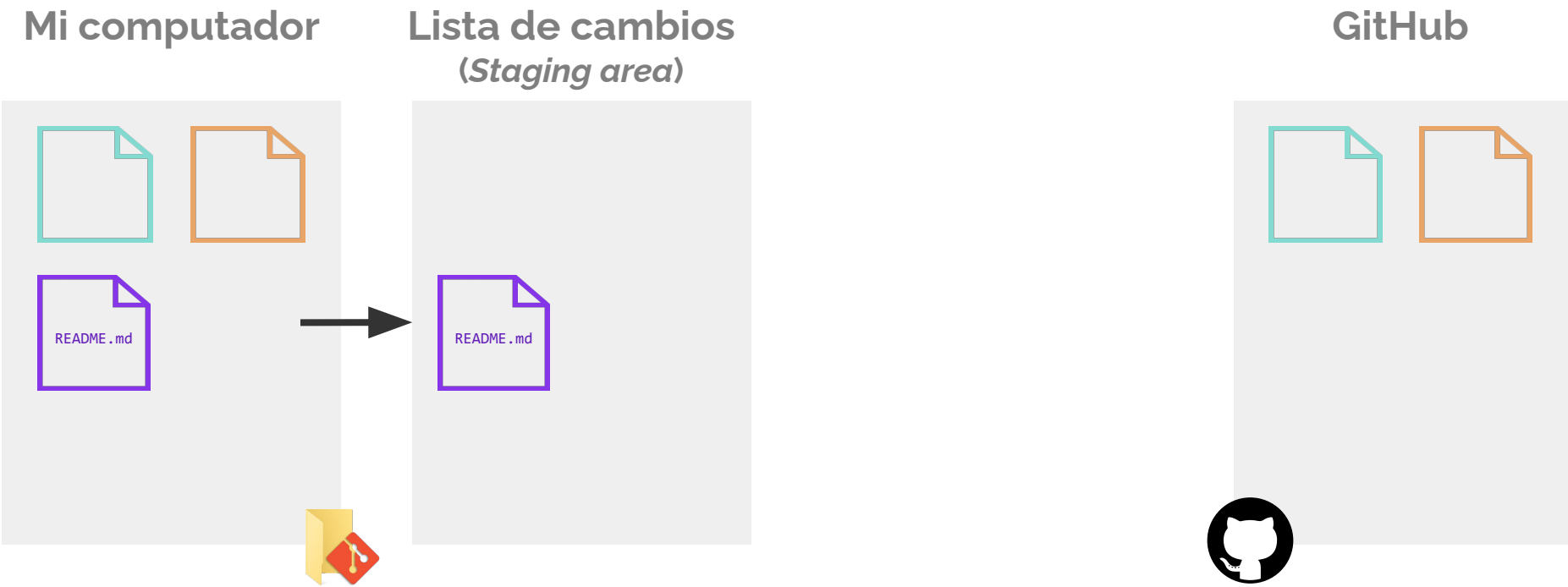



```
git add README.md
```

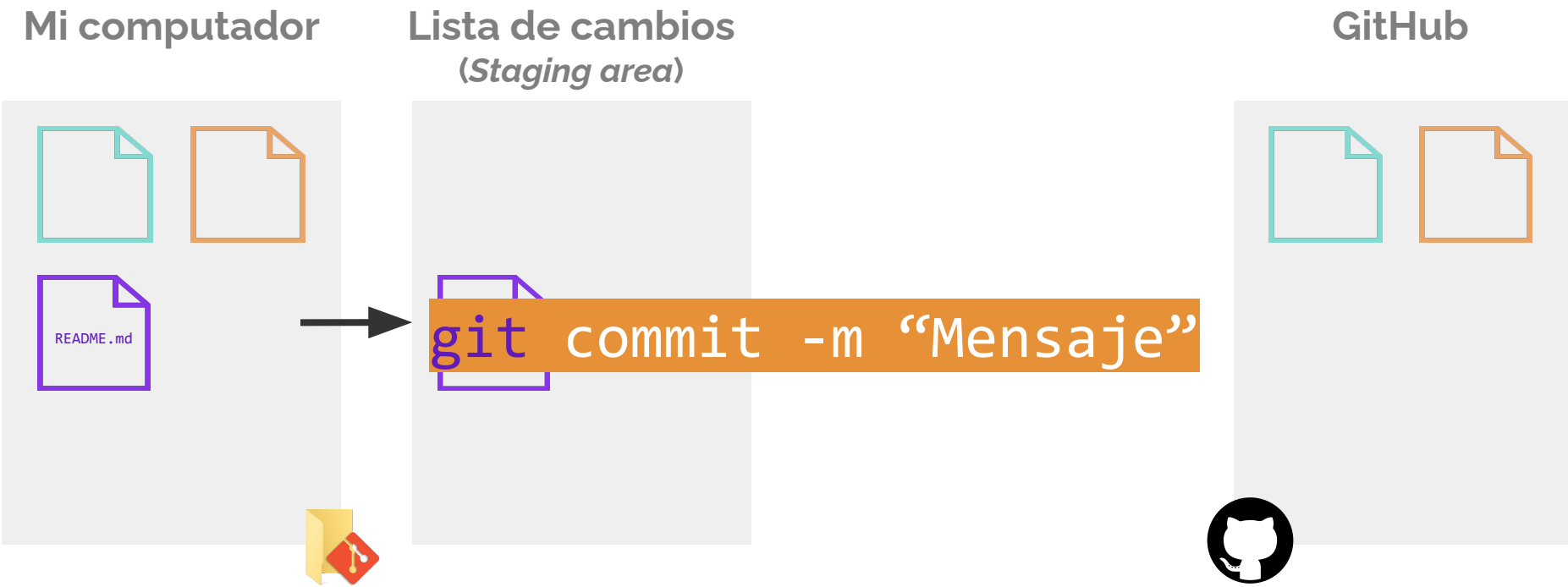
GitHub



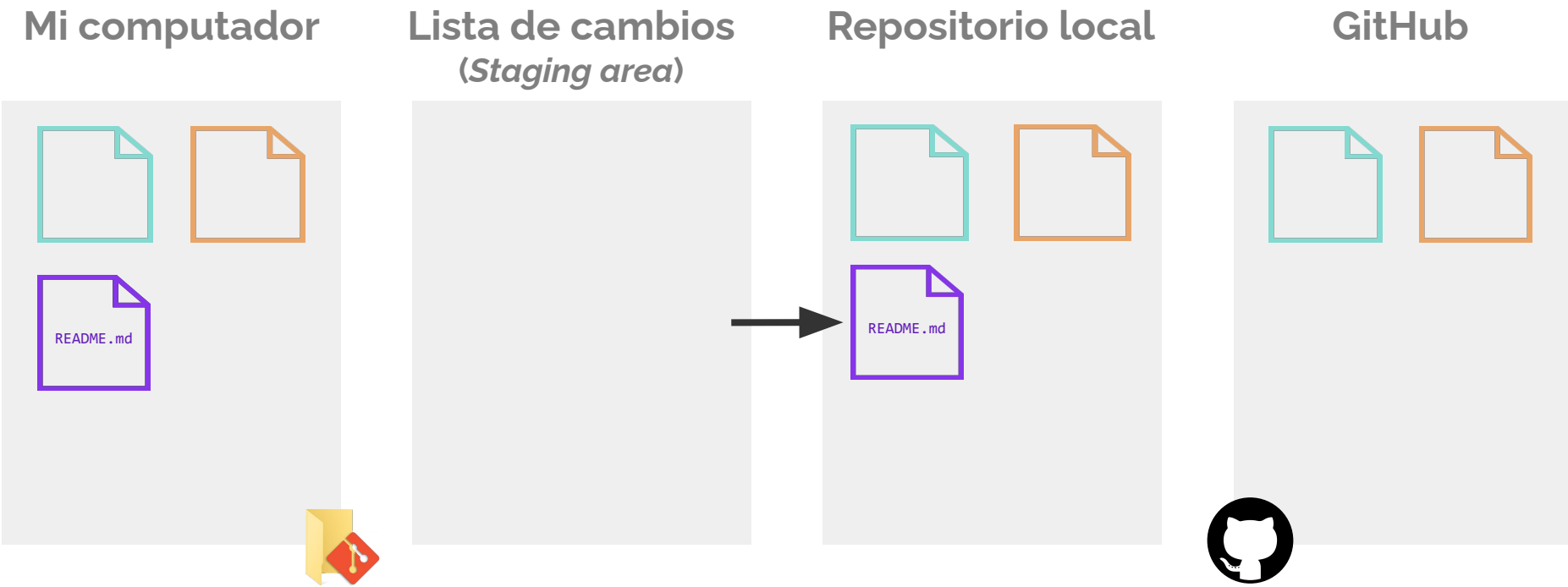
¿Qué acabamos de hacer?



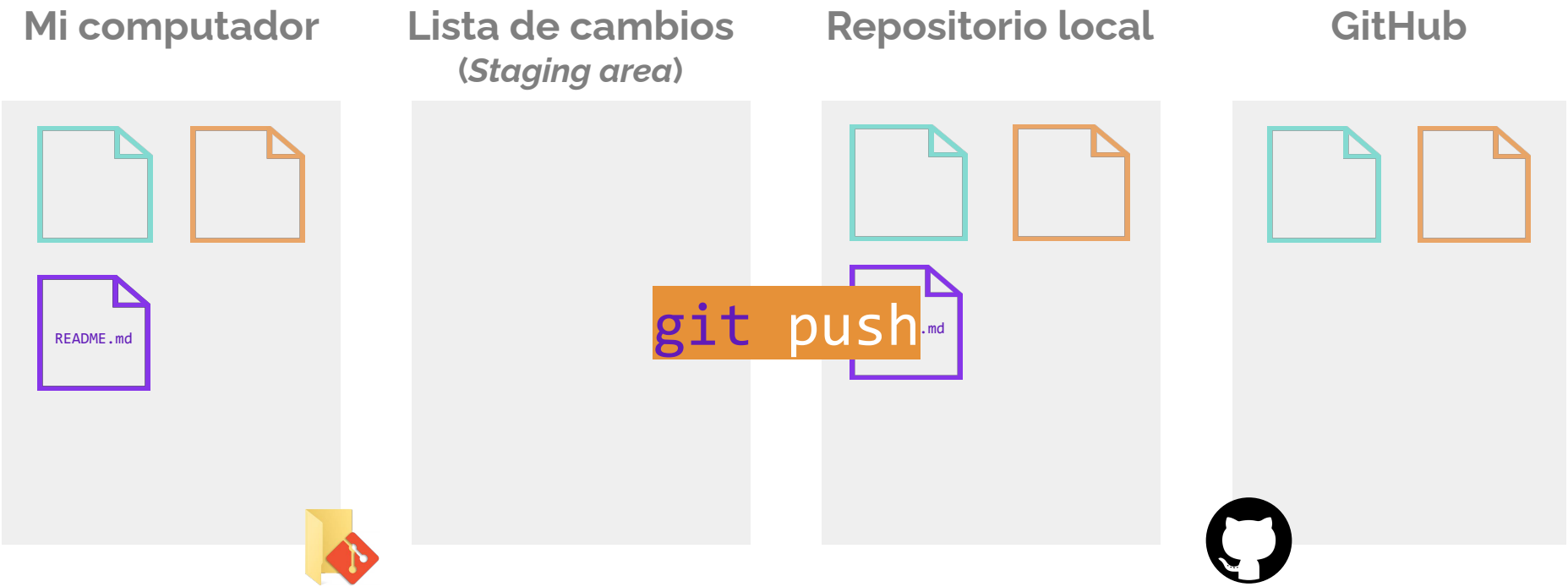
¿Qué acabamos de hacer?



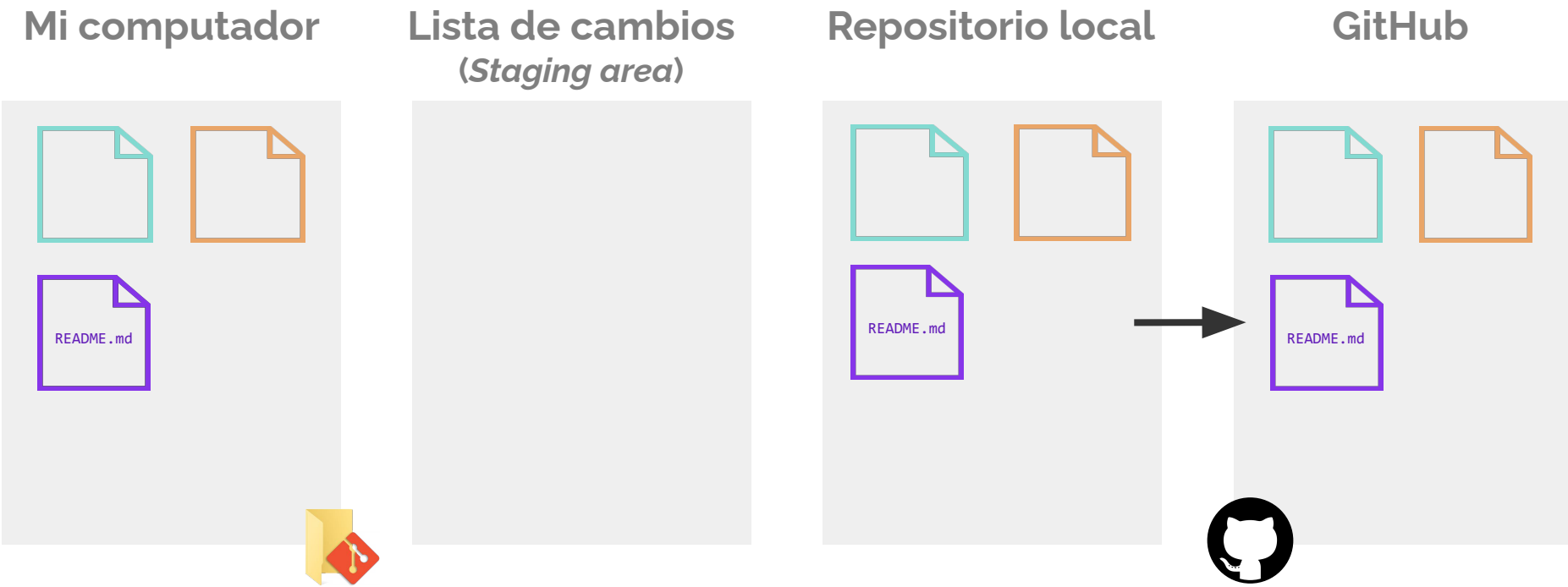
¿Qué acabamos de hacer?



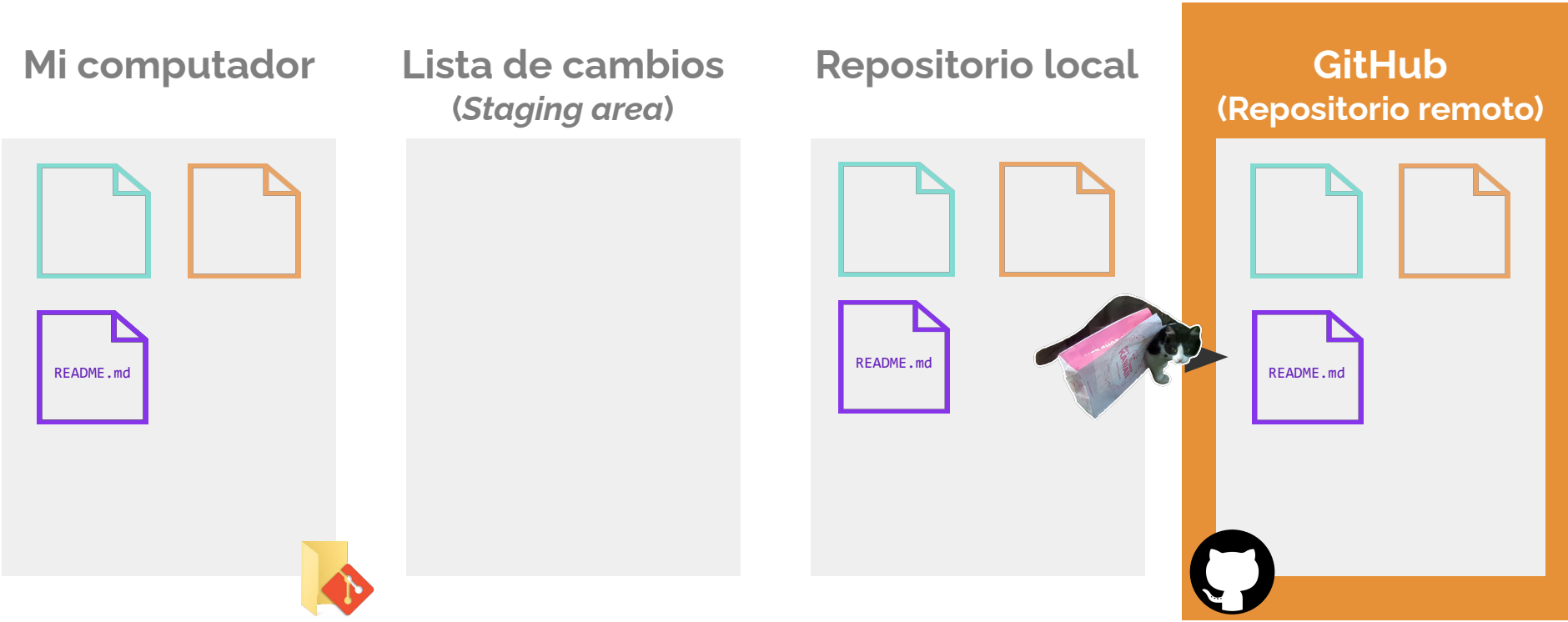
¿Qué acabamos de hacer?



¿Qué acabamos de hacer?



¿Qué acabamos de hacer?



Siempre hagan *add*, *commit* y *push*

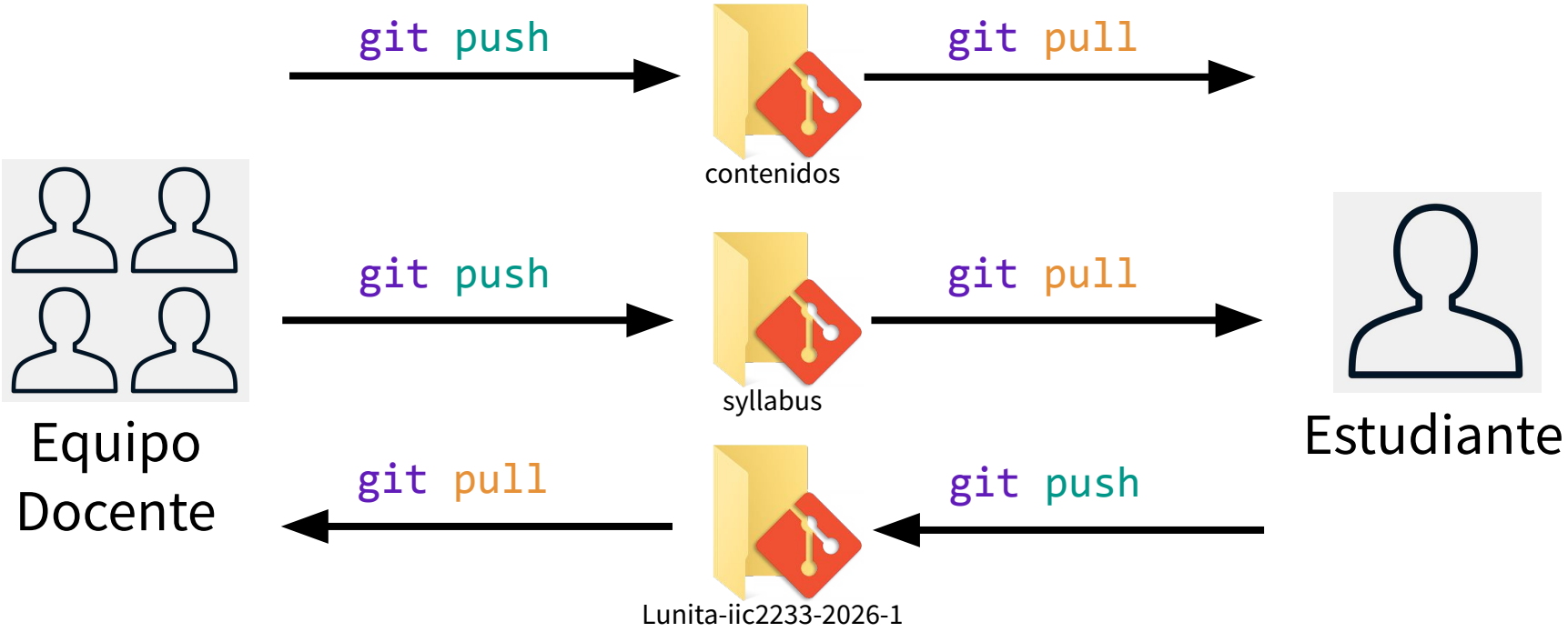
¿**Cuándo** hacerlo?

- Cada vez que avancen en algo importante de su actividad o tarea.
- Si llevan programando más de media hora.
- Cuando paren para hacer otra cosa.

¿**Por qué** hacerlo?

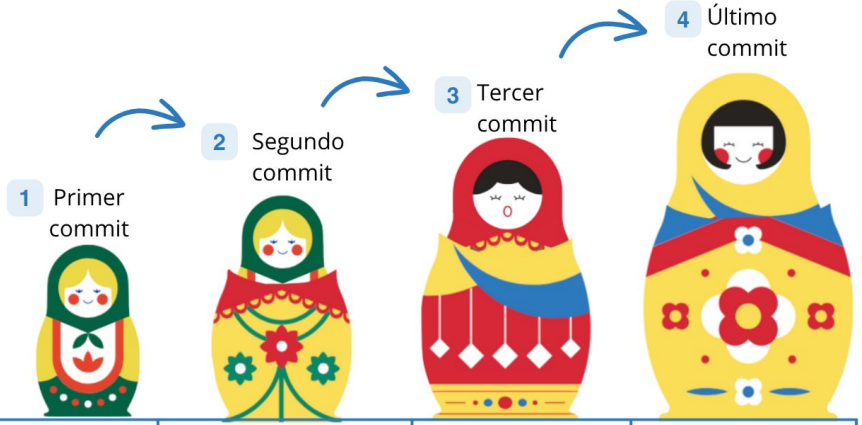
- Tener su trabajo en GitHub es una copia de seguridad.
- *Shit happens*:
  - Accidentes con líquidos.
  - Robos en Deportes.
  - Fallas de *hardware* o *software*.
  - Cortes de internet.
  - Echar a perder la tarea.
  - Y muchas otras cosas.

El flujo durante el semestre



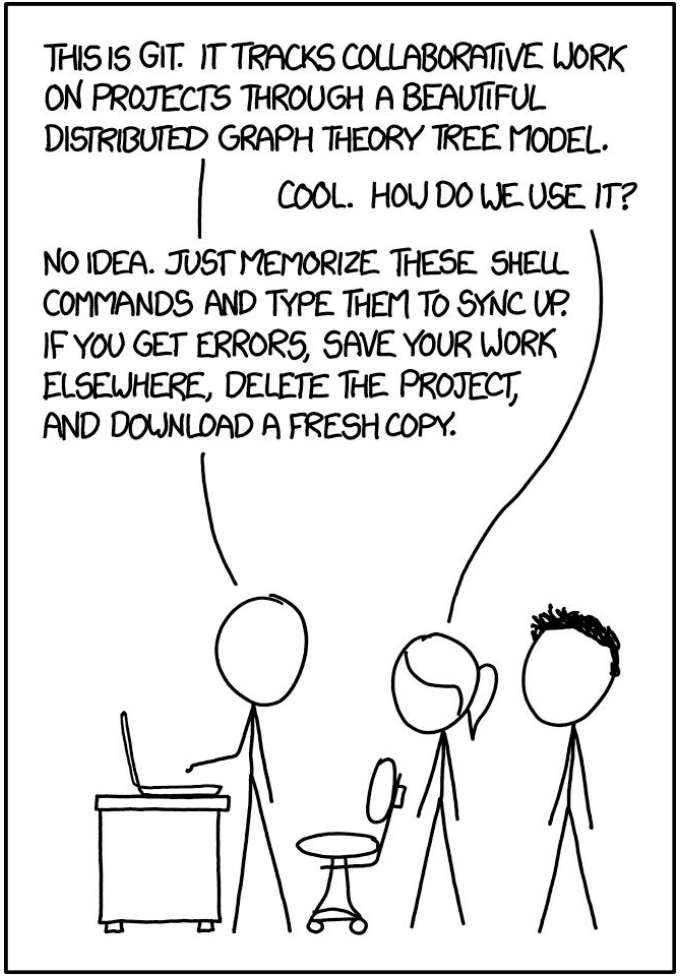
Entendiendo un *commit*

- Cada *commit* **guarda una foto completa del proyecto**, incluyendo todos los cambios anteriores.
- Es como una muñeca rusa: el **último *commit* contiene una referencia a todos los commits anteriores**.



Commit	Primer	Segundo	Tercer	Cuarto
Cambios realizados	Se agrega el archivo main.py	• Se agrega el archivo utils.py • Se importan las funciones de utils.py en el archivo main.py	Se corrige un error de una función en utils.py	Se agrega README.md

Un último consejo...



Pregunta de Evaluación Escrita

Veamos una pregunta de Evaluación Escrita

Tema: Entorno trabajo y git (Midterm 2025-2)

1. En Git, ¿qué contiene un *commit*?

   A) Solo los archivos modificados hasta ese momento.

   B) Solo los archivos nuevos agregados hasta ese momento.

   C) El estado completo del proyecto hasta ese momento.

   D) Solo los archivos eliminados hasta ese momento.

   E) La configuración del repositorio en ese momento.

Veamos una pregunta de Evaluación Escrita

Tema: Entorno trabajo y git (Midterm 2025-2)

1. En Git, ¿qué contiene un *commit*?

   A) Solo los archivos modificados hasta ese momento.

   B) Solo los archivos nuevos agregados hasta ese momento.

   C) El estado completo del proyecto hasta ese momento.

   D) Solo los archivos eliminados hasta ese momento.

   E) La configuración del repositorio en ese momento.

¿Qué es Inteligencia Artificial?

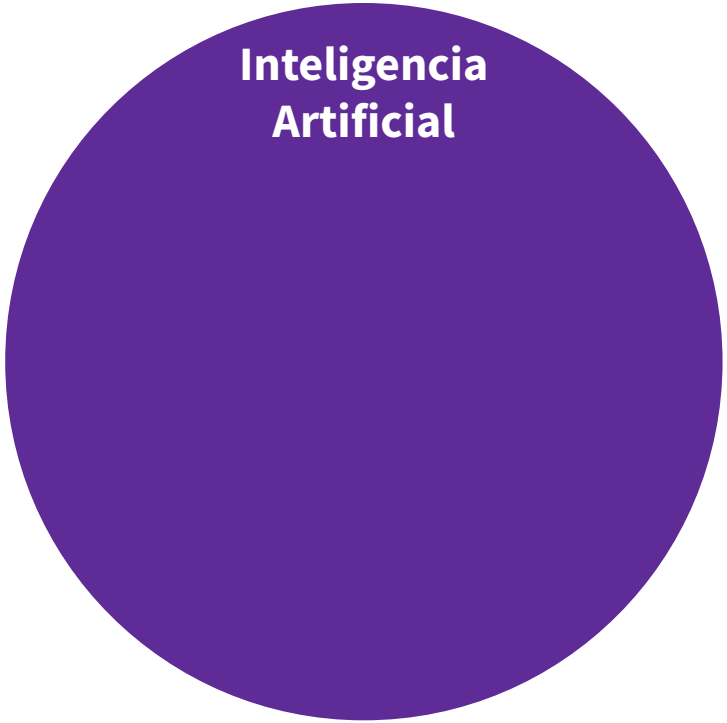
Basado en las clases de Antonio Ossa (2025-2)

¿Qué es la IA que conocemos hoy en día?

El campo de **Inteligencia Artificial (AI)** usa ciencia de la computación y datos para resolver problemas usando máquinas.

Ejemplos:

- Programas que juegan ajedrez
- Algoritmos de búsqueda
- Programas de lógica condicional
- Predicción de estructuras tridimensionales

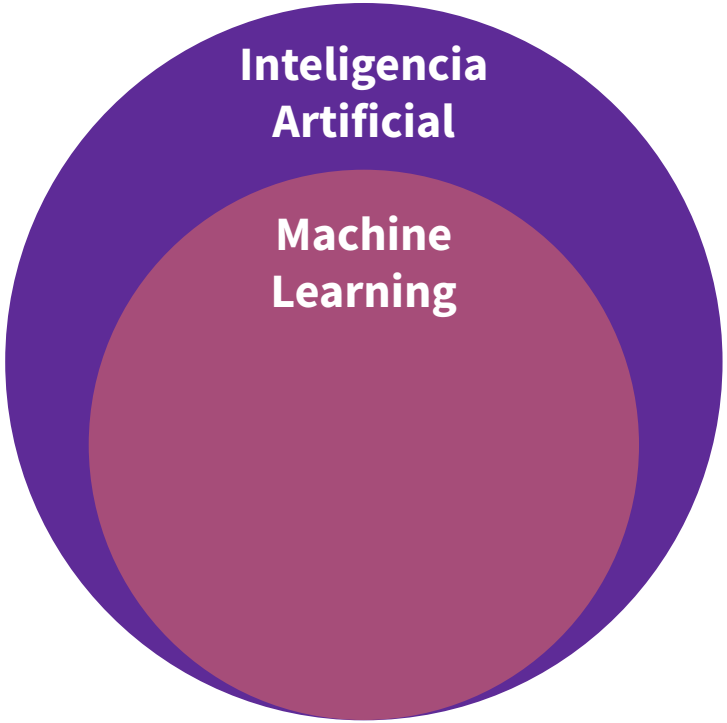


¿Qué es la IA que conocemos hoy en día?

***Machine Learning* (ML)** es el estudio de sistemas computacionales que aprenden y se adaptan automáticamente a partir de experiencia, sin ser explícitamente programados.

Ejemplos:

- Sistemas de recomendación
- Modelos de predicción sobre sensores
- Algoritmos de clusterización

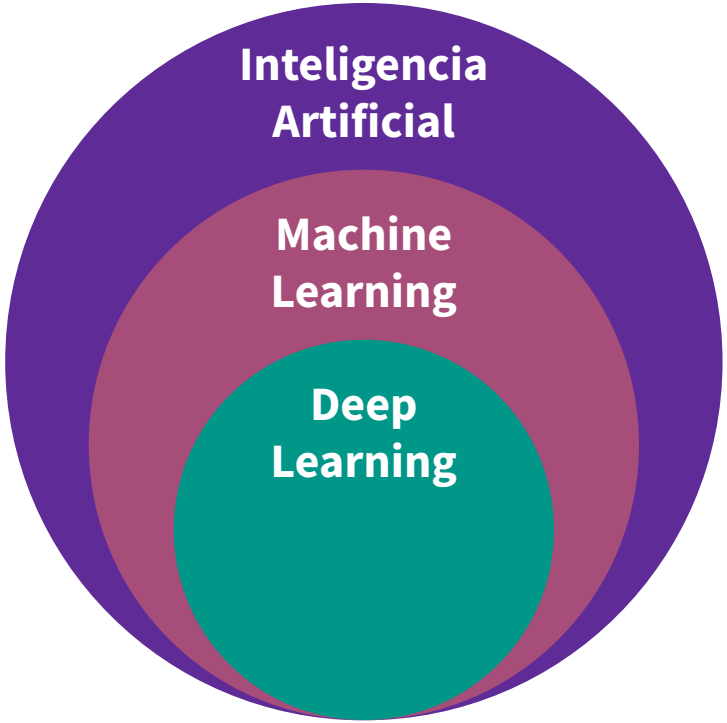


¿Qué es la IA que conocemos hoy en día?

***Deep Learning* (DL)** es una técnica de *Machine Learning* (ML) en la que se construyen capas de algoritmos y unidades de procesamiento (neuronas) dentro de una red neuronal artificial (ANN o NN).

Ejemplos:

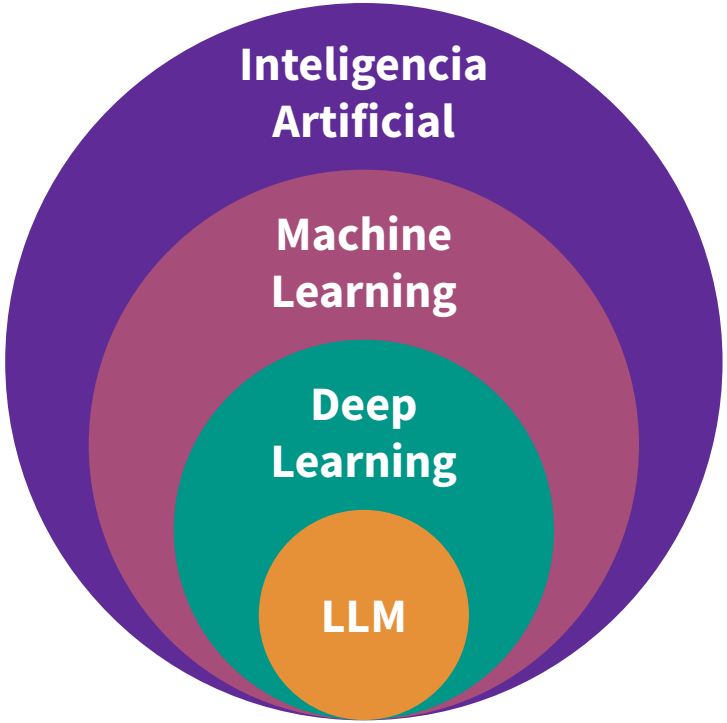
- Detección de objetos
- Segmentación de imágenes
- Vehículos autónomos
- Chatbots



¿Qué es la IA que conocemos hoy en día?

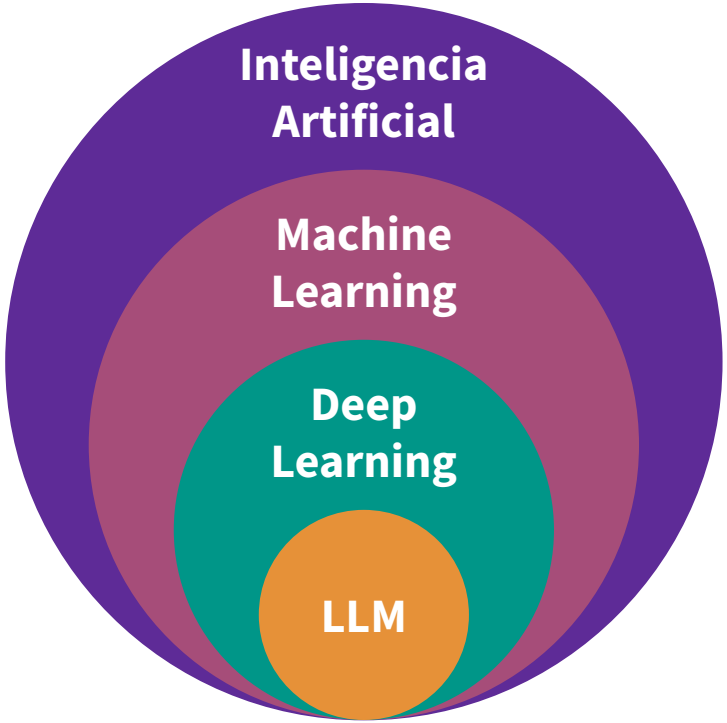
Lo que conocemos hoy en día como Inteligencia Artificial (IA) es un tipo de programa llamado ***Large Language Model* (LLM)**.

Son programas que puede reconocer y generar texto similar al que podría generar un humano.



¿Qué es la IA que conocemos hoy en día?

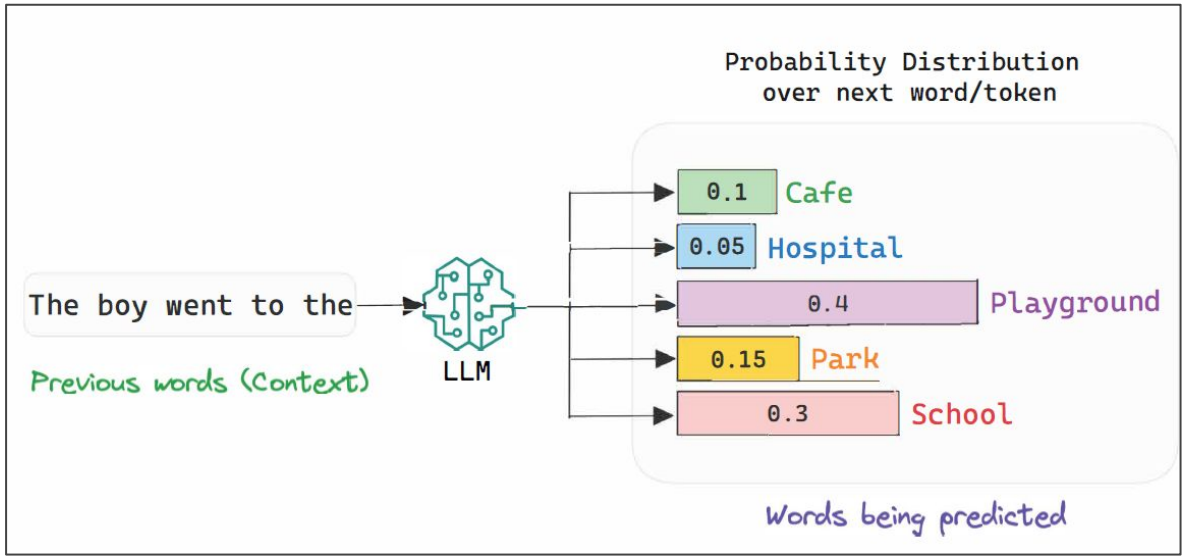
Es un modelo que ha sido entrenado con suficientes ejemplos de texto como para reconocer e interpretar el lenguaje humano y otros tipos de datos complejos. Comúnmente, se utiliza texto recolectado de Internet.



¿Cómo funcionan los modelos de lenguaje?

Todos los modelos de lenguaje siguen la misma idea:

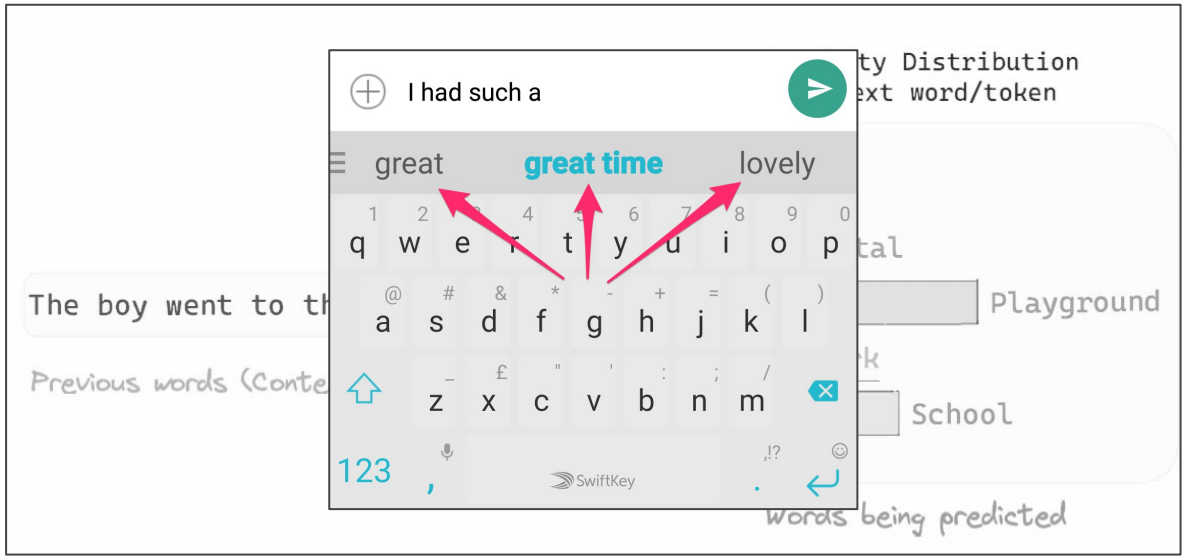
Dada una frase, aprender a predecir la distribución de probabilidades de siguientes palabras



¿Cómo funcionan los modelos de lenguaje?

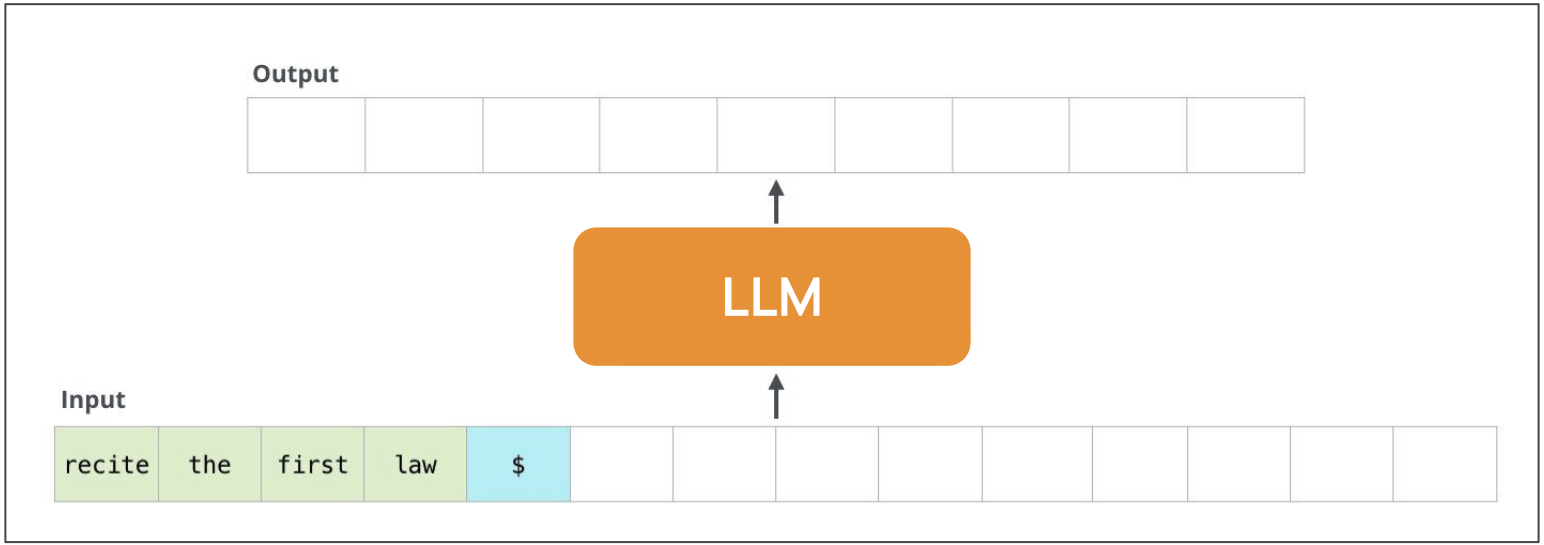
Todos los modelos de lenguaje siguen la misma idea:

Dada una frase, aprender a predecir la distribución de probabilidades de siguientes palabras



¿Cómo funcionan los modelos de lenguaje?

Luego, selecciona alguna de las palabras predichas (según la distribución de probabilidad), actualiza el contexto y vuelve a predecir…



¿Cómo funcionan los modelos de lenguaje?

Debido a la naturaleza de los modelos de lenguaje, estos **no contienen conocimiento real** sino que solo emiten texto que busca mantenerse coherente y aparentar que ha sido escrito por un humano.

Usar
Inteligencia Artificial

🚀 Beneficios

1. **Mayor productividad**
   - Genera código rápidamente a partir de descripciones.
   - Reduce tiempo en tareas repetitivas.
   - Permite prototipar ideas simples en minutos.
2. **Aprendizaje acelerado**
   - Explica conceptos de programación.
   - Muestra ejemplos prácticos de cómo implementar soluciones.
   - Puede ayudar a aprender nuevos lenguajes o *frameworks* más rápido.

🚀 Beneficios

3. **Depuración y análisis**
   - Puede detectar posibles errores o vulnerabilidades.
   - Sugiere mejoras en el código.
   - Ayuda a interpretar errores complejos.

4. **Automatización de tareas**
   - Generación de documentación.
   - Creación de pruebas unitarias.
   - Conversión de código entre lenguajes.

🚀 Beneficios

5. **Accesibilidad**
    - Reduce barreras de entrada para crear *software*.

⚠️ Riesgos

1. **Código incorrecto o inseguro**
   - **Puede generar código ejecutable pero es incorrecto o inseguro.**
   - Riesgo de introducir vulnerabilidades sin darse cuenta.
2. **Dependencia excesiva**
   - **Programadores pueden dejar de desarrollar habilidades fundamentales.**
   - Dificultad para resolver problemas sin IA.

⚠️ Riesgos

3. **Falta de contexto**
   - La IA no siempre entiende el sistema completo.
   - Puede proponer soluciones incompatibles con la estructura del proyecto.

4. **Problemas de propiedad intelectual**
   - Existe debate sobre si el código generado puede contener partes derivadas de repositorios existentes.

⚠️ Riesgos

5. **Fuga de información**
   - Si se envía código privado a la IA, puede haber riesgos de seguridad o confidencialidad.

6. **Mantenimiento difícil**
   - Código generado automáticamente puede ser:
     - Difícil de entender.
     - Inconsistente con estándares del proyecto.

🎈 Recomendaciones

1. Al preguntar, **entregar la mayor cantidad de contexto posible**:
   - Lenguaje de programación y versión
   - Librerías que sí o no se pueden utilizar
   - Archivos, funciones y clases que ya existen

   Por lo general, entre más contexto tenga la IA, mejor será su respuesta.

   Además, **ser explícito sobre lo que sí y no puede hacer** marca una gran diferencia en el resultado.

🎈 Recomendaciones

No es lo mismo pedir:

Escribe un código que calcule el promedio ponderado de 4 tareas.

Utilizando **Python3.12** y las **librerías numpy y pandas**, genera un código que permita **calcular el promedio ponderado** de 4 notas.

Las **notas vienen** en un archivo CSV y pueden ser identificadas a partir de los *headers* T0, T1, T2, T3; la ponderación de cada una de estas notas es 1, 2, 2, y 3 respectivamente.

La **nota resultante se debe** agregar al CSV en la columna "Promedio Tareas".

🎈 Recomendaciones

2. Cuidado con las **alucinaciones de la IA**.

   Muchas veces las **respuestas** entregado pueden ser **falsas, incorrectas, o no tener sentido**. Recuerden siempre comprobar su veracidad:

   - Pidiendo fuentes y comprobando su contenido.
   - Revisar el código entregado y realizar un **proceso de investigación, integración y adaptación**.

💡 Consideraciones

1. Si bien el **código entregado por la IA no es de su autoría**, desde el momento en que entregan ese código en una evaluación **ustedes son responsables** de su contenido.

   La entrega de código alucinado, malicioso o que haga referencia a elementos prohibidos, es considerado una **falta a la integridad académica y los objetivos del curso, por lo que será sancionado**.

💡 Consideraciones

2. El que se permita el uso de IA en la resolución de tareas, **no implica que lo puedan utilizar en las otras evaluaciones del curso**.

   El uso de IA en la resolución de Evaluaciones Escritas, Actividad o Experiencias, es considerado una **falta a la integridad académica y será sancionado**.

💡 Consideraciones

3. Mantengan un **registro del los usos de IA**.

   Cuando termine el desarrollo de una Tarea, deberán **rellenar un formulario indicando dónde, cómo y para qué utilizaron IA** para resolver la evaluación. Además, podrán ingresar los *props* que utilizaron.

   El **no responder** este formulario de uso de IA implicará un **descuento en la evaluación**.

   El **omitir información o entregar información falsa** es considerado una **falta a la integridad académica**.

🤓 Alternativas a la IA

Google sigue siendo una excelente herramienta, solo hay que saber cómo utilizarla bien:

1. Existen comandos para buscar de forma más eficiente.
   (Recomendamos investigar sobre sobre “”, *, -, AND, OR)
2. Buscar en inglés te da acceso a una mayor cantidad de resultados.
3. Agregar información contextual a la búsqueda.
   (Indicar la versión de Python o la librería puede hacer una gran diferencia)
4. Eliminar información innecesaria para la búsqueda.

🤓 Alternativas a la IA

Python y sus librerías tienen **documentación oficial**. La puedes revisar para aprender cómo utilizar sus funciones y clases.

- Funciones Built-ins
- Módulo random
- Glosario de Python

Programación Avanzada
IIC2233 2026-1

Jaime Middleton - Pablo Araneda - Antonio López - Tamara Vidal - Daniela Concha

Comentarios AC1